

Cybersecurity

Microservices and Web Application Architecture

Lesson 13.2



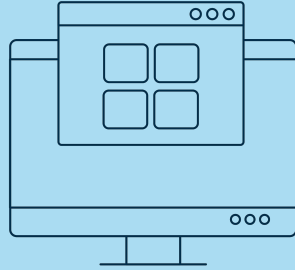
Learning Objectives

By the end of today's class, you will be able to:

- 1 Understand how microservices and architecture work to deliver more robust, reliable, and repeatable infrastructure as code.
- 2 Define the different services within a LEMP stack.
- 3 Describe how relational databases store and retrieve data.
- 4 Create SQL queries to view, enter, and delete data.
- 5 Use Github Copilot to create a front end.



Welcome



Application Structure

Application Structure

Today, we'll take a deeper look at the infrastructure behind web applications.

01

Components that make up web architecture and their software "stacks"

02

How web application components can be deployed using containers

03

How databases work to understand how web apps store and transmit data

04

How AI can be used to create websites easily using GitHub Copilot



Demonstration Scenario



Andrew, a developer, uses a browser to manage a public-facing application (the Employee Directory App), where he retrieves and updates different kinds of employee lists of his company. The app is set up so that Andrew can add, remove, and view employees.

This app needs the following components to function:

- Front-end server
- Back-end server
- Database



Application Structure

Components of a Typical Web App

Front-end server

Responsible for displaying webpages and styling them in a readable format. This server also receives and responds to HTTP requests.

Back-end server

Executes business logic and writing or reading corresponding data to and from a database. The back-end server knows how to interact with the database depending on the specific request received.

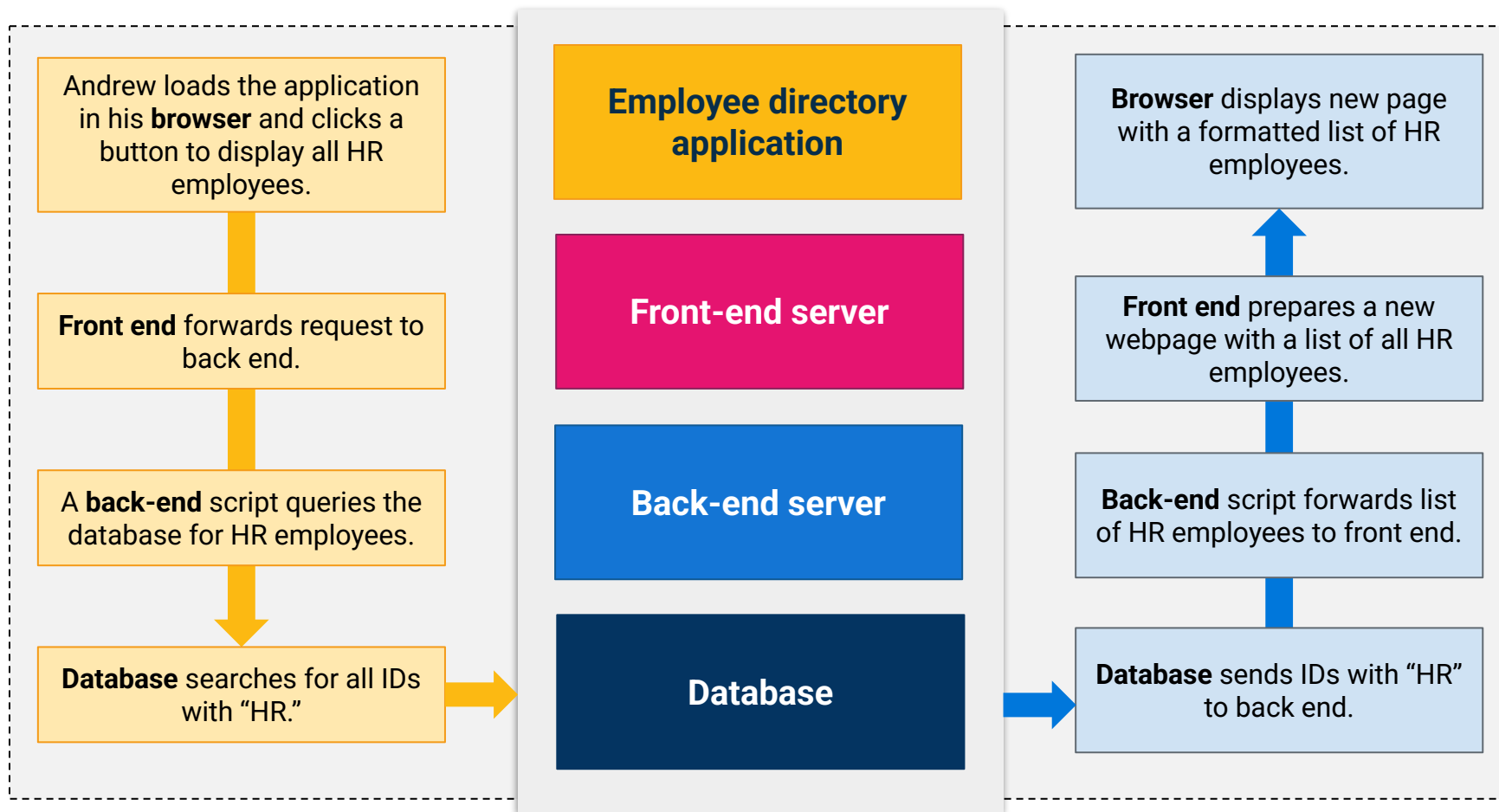
Database

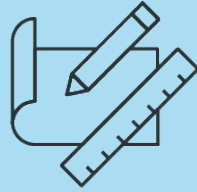
Stores information on employees, such as their employee IDs and names. Critical and sensitive information is found in databases.

Now that we know the main components of a web application, we want to know what happens behind the scenes when a user interacts with a web application, or more specifically, makes an **HTTP request** to the server.

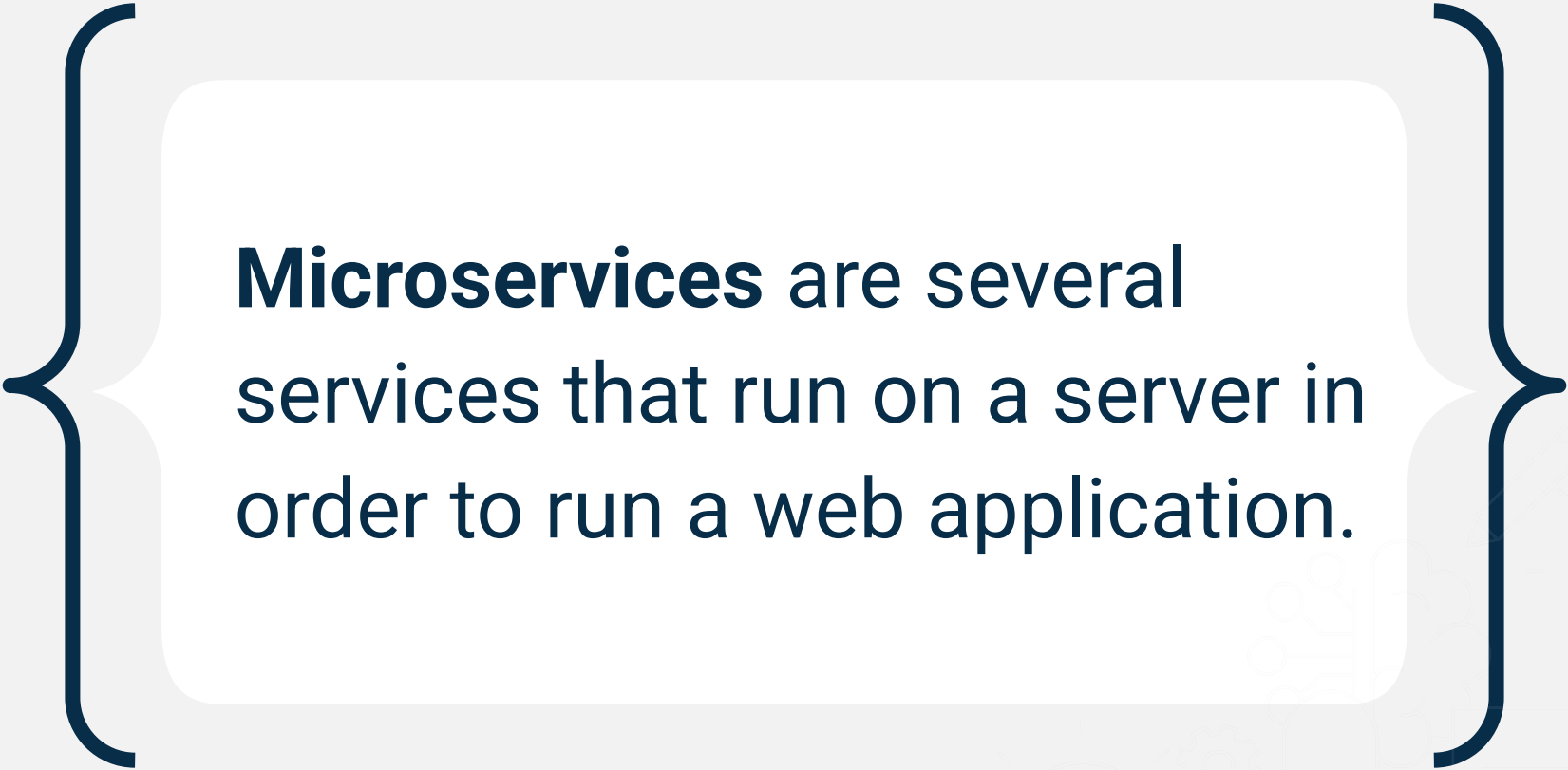


Information flows Between Application Components

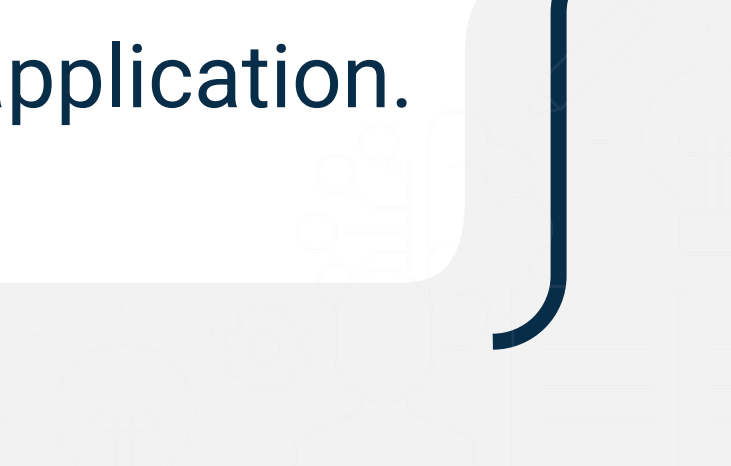




Modern Architecture



Microservices are several services that run on a server in order to run a web application.





Modern Architecture

Web applications today are composed of many microservices

One of the first steps in creating microservices is separating components in individual machines to run a “**service.**”

A **service**, such as the front-end server, is really just an operating system, running a minimum amount of software to serve one main purpose.



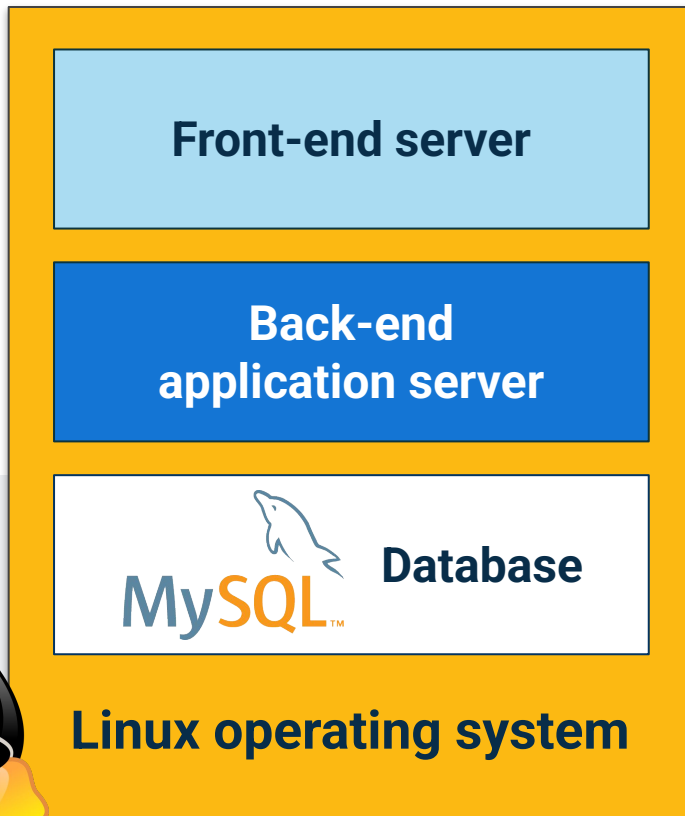
Web Application Architecture

Each individual machine requires:

- An operating system to run on in order to run the service's software and store its configurations.
- The software that runs a specific set of instructions for that individual component of the web application.

For example:

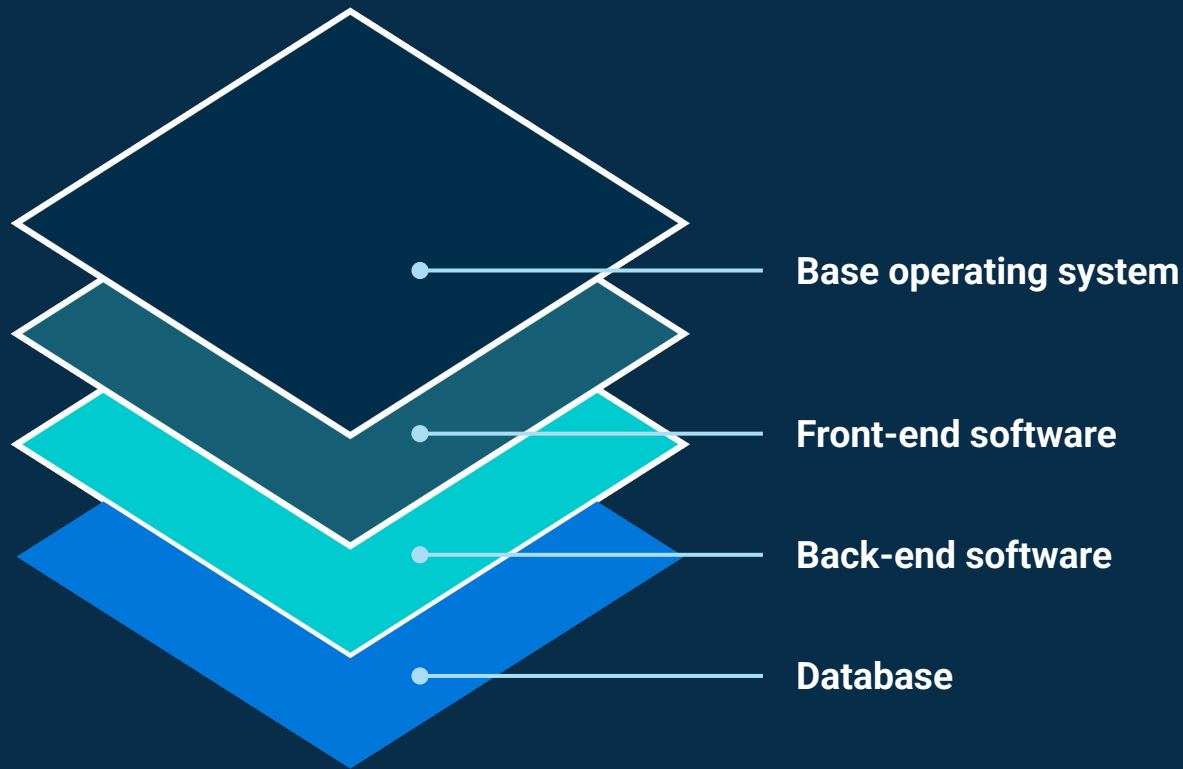
A database service runs Linux as its operating system and has a MySQL database installed on it.



Introducing Stacks

What open-source and commercial software do we need to set up in order to create our web application components?

Any combination of a base operating system, front-end software, back-end software, and database is called a **web stack**.



Introducing Stacks



Base operating system



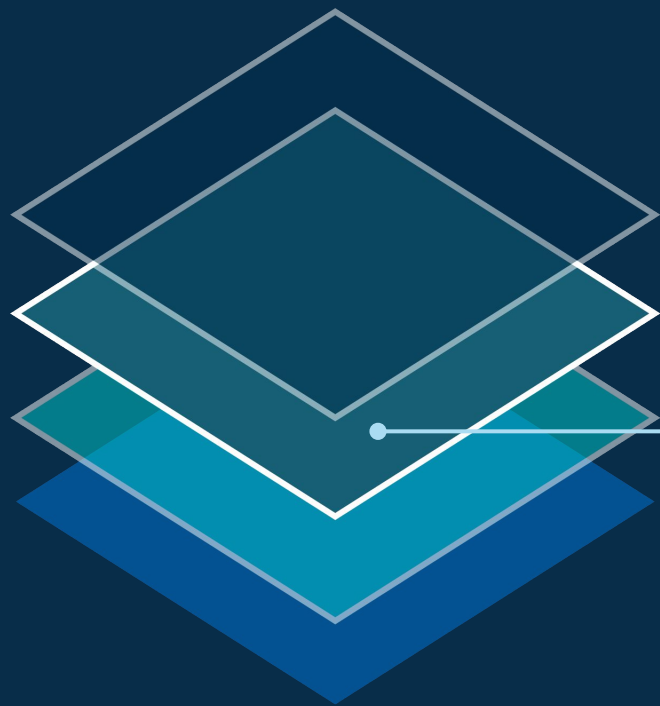
Red Hat

ubuntu 



SUSE

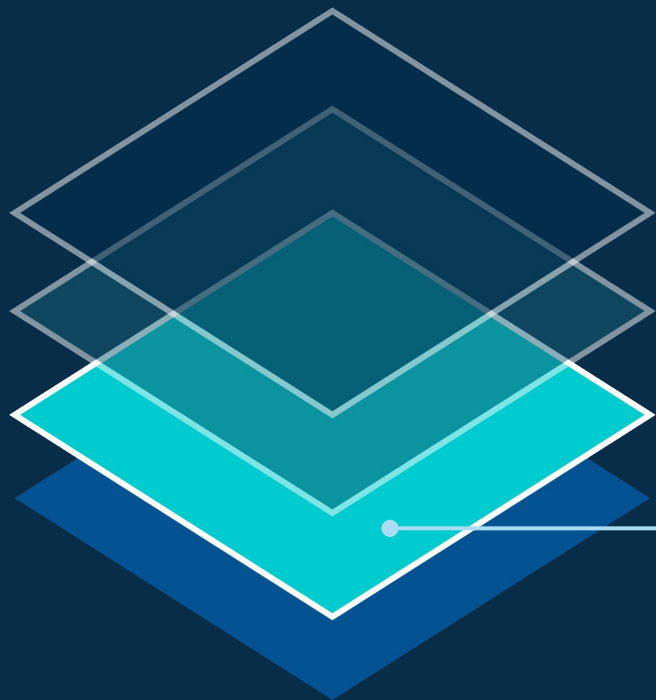
Introducing Stacks



Front-end software



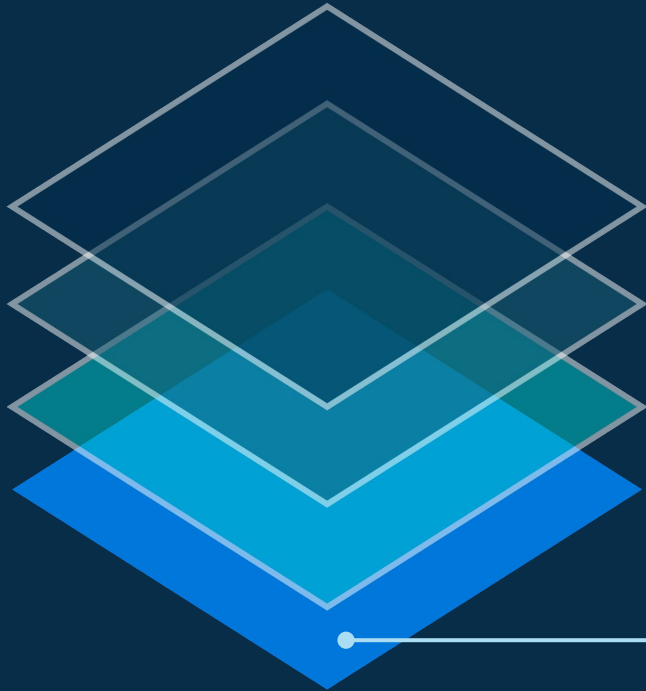
Introducing Stacks



Back-end software



Introducing Stacks



Database



LAMP Stack

A popular example of a stack is the **LAMP** stack.

Linux as the base operating system

Apache to handle the front-end HTTP
requests & responses

MySQL for storing data

PHP for back-end transformation of front-end
requests and responses to database queries



ubuntu



APACHE®



MySQL™



LAMP Stack

The LAMP stack is considered the tried and true web stack, with a long history of use in architecting web applications.

The following sites either still use or originally used the LAMP stack:



facebook

tumblr.



WIKIPEDIA

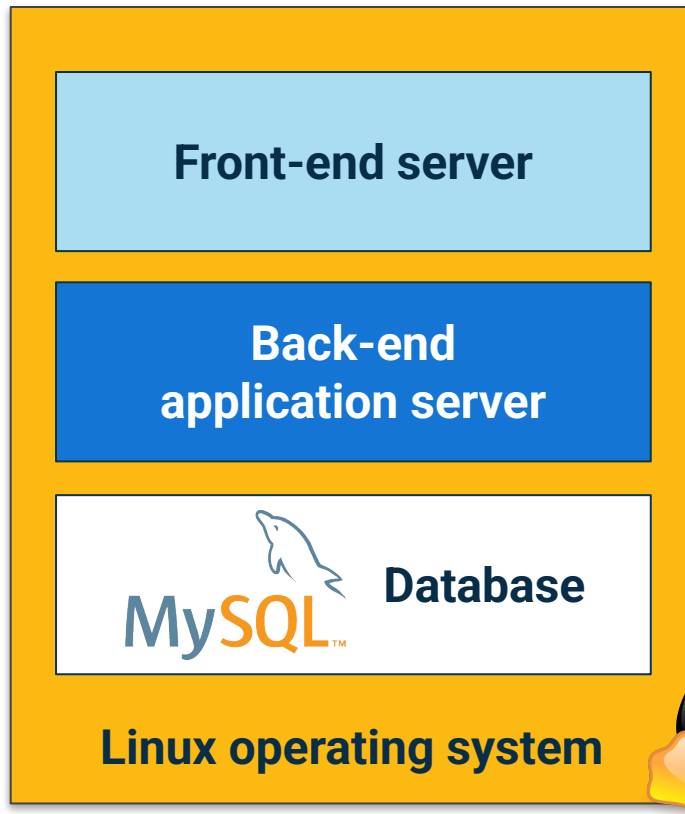


More Stacks

These days, there are many more web stacks, which use different variations of software.

But what stacks should we use.
for our own web application?

Let's take a look at some options.



Common Web Architecture: Operating Systems

Let's start by comparing the different **operating systems** available for our services.



The **Alpine Linux** is a super lightweight Linux distribution that has the minimum requirements to run containers.

Other container-friendly operating systems include most other Linux distributions, such as Ubuntu, and more recent versions of Windows Server. The following base operating systems are used for web applications:

Common Linux distros such as:



Windows Server 2016 and
higher



Common Web Architecture: Front-end Server

Our front-end server handles the HTTP requests and responses. It is usually some form of HTTP server running on a lightweight containerized operating system. The front-end server is the part of a web application that renders the visual components of a web application.

Primary front-end web design languages:



Specific examples of front-end servers:



Common Web Architecture: Database Server

Data for the application is housed in the database server. Remember that there could be multiple databases, some for storing sessions and cookies and some for storing business logic like a company's employee directory.

Specific examples of databases:



Honorable Mention: The WIMP Stack

There is also a Microsoft-based variation called the **WIMP** stack that uses the following:

Windows operating system

Microsoft's **I**IS web server in place of Apache/Nginx

Database such as **M**icrosoft SQL Server

Back-end language such as **P**owerShell



Like the LAMP stack, we won't use WIMP in class, but it is good to know that Microsoft-based web stacks are available.

LEMP Stack

The web software stack that we'll use for today's demos and activities consists of:



L

Linux

The standard underlying operating system used for web applications. However, we will use it with containers.



E

Nginx

We'll use nginx for our front-end HTTP server, instead of Apache, mainly for its speed. While Apache is considered to have more features and is more compatible with back-end languages such as Java, nginx is well known for its performance.



MariaDB

M

MariaDB

Unofficially the "other" MySQL, it is a standard relational database that will handle our data storage.



P

PHP

Our pages will be generated with data pulled by PHP scripts. We will not need to manage or work with any PHP directly today, but know that it is acting as our back-end language.



We just covered all the software that we want to use for our web application, but what about the **hardware**?

Microservices Infrastructure with Containers

Microservices require lightweight environments, because:



Our microservices need to be rapidly deployable.



We need to be able to scale our microservices to meet growing demand.



Developers and maintainers need to be able to replicate their own copies of these microservices locally.



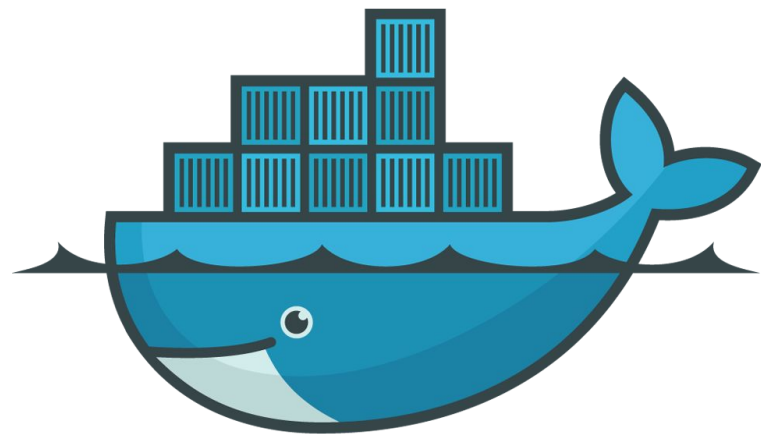
Having full-sized virtual machines for each service would require more resources and be more costly.



Can anyone think of a technology we've covered in class that can create **isolated virtual environments**?

Microservice Infrastructure with Containers

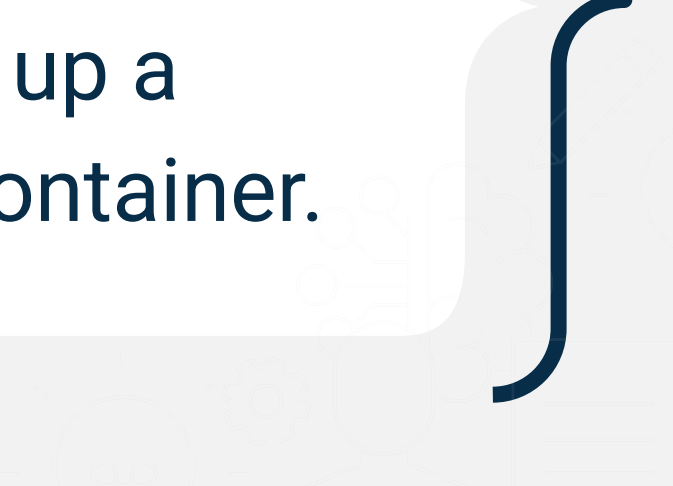
- Containers run as isolated, virtual operating systems that dynamically allocate resources depending on needs.
- This is unlike regular virtual machines, where all hardware is virtualized.
- Docker is the most popular container platform. For the rest of this class, we will refer to Docker's implementations of containers.



docker



Containerization is the process of packaging all the requirements to set up a microservice as a container.



How a Container Becomes a Microservice

Containerizing a microservice requires the following steps:

01

Declares a base operating system for the microservice to run on.

02

Stages the microservice's source code to the container.

03

Sets a command that launches the microservice and all other needed software when the container is deployed.

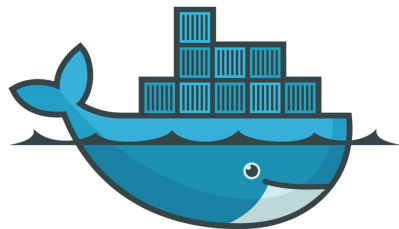
Dockerfile

Containerization can be declared in a text file called a **Dockerfile**.

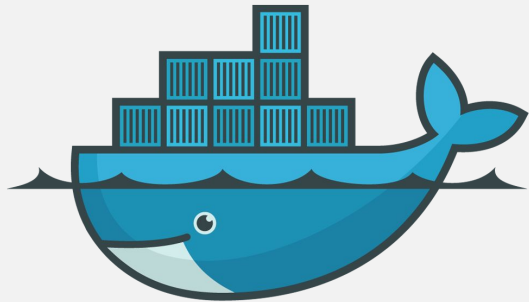
```
#####  
# Employee Directory Application  
#####  
# Required Base OS  
FROM ubuntu:14.04  
...[truncated]
```

Dockerfiles contain all the configuration needed for a container in one file.

This is similar to an Ansible configuration file, which contains everything needed to configure a host with the **command**:
ansible-playbook
your-playbook.yml.



A unique Dockerfile is created for each microservice.



docker

Product
Directory
app



Dockerfile

```
#####  
# Product Directory Application  
#####  
# Required Base OS  
FROM ubuntu:14.04
```

Product
API



Dockerfile

```
#####  
# Product API  
#####  
# Required Base OS  
FROM fedora:31
```

Database



amazon

Dockerfile

```
#####  
# AMAZON Database  
#####  
# Required Base OS  
FROM centos:8
```



Intro to **Databases**



Let's recap



Recap

Let's recap everything we've learned so far today:

01

How to describe web application architecture

02

Defining specific web application architectures

03

How to deploy and test that a container set is running and functional

The Importance of Databases

Databases contain the persistent data that flows through the web application.

The data in the database can include, but isn't limited to:

- The usernames, telephone numbers, and email addresses of customers
- The session and cookie IDs of site visitors
- Personally identifiable information, such as the full names, dates of birth, credit card numbers, and social security numbers of customers



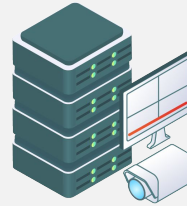
File server



Database server



Proxy server



Video server



VoIP server



Virtual server



Virtual private network server



File Transfer Protocol server



Mail server

The Importance of Databases

It is absolutely crucial that security professionals have a strong understanding of how databases work in web applications.



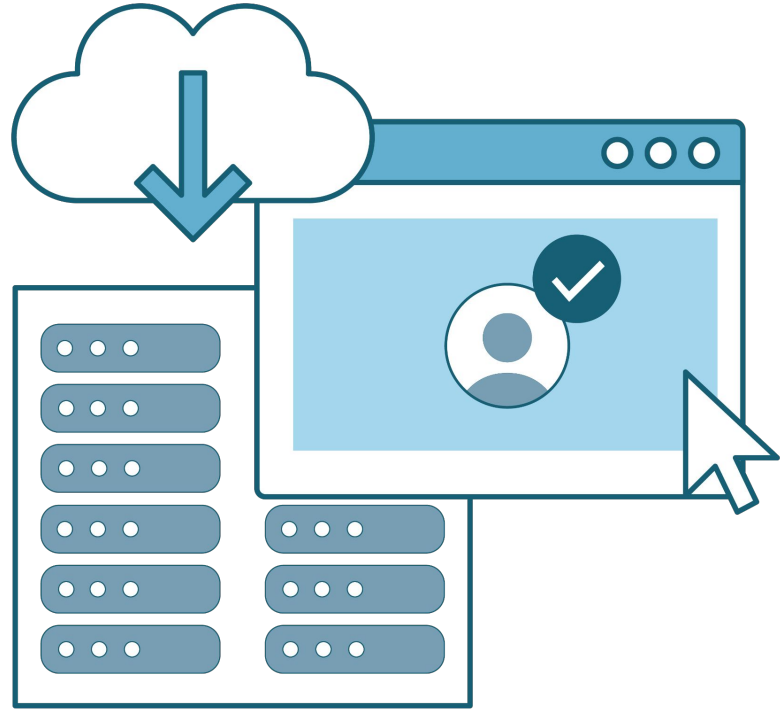
We'll examine the threats and vulnerabilities of databases more closely later in the course.



Databases Overview

Databases are used to store large amounts of data.

Databases are typically kept separate from the other components of a web application.



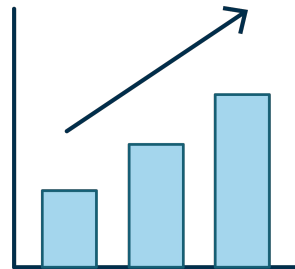
Databases are typically kept separate

Why we separate databases from other web apps:



Security

Compromise of one machine does not imply compromise of the other.

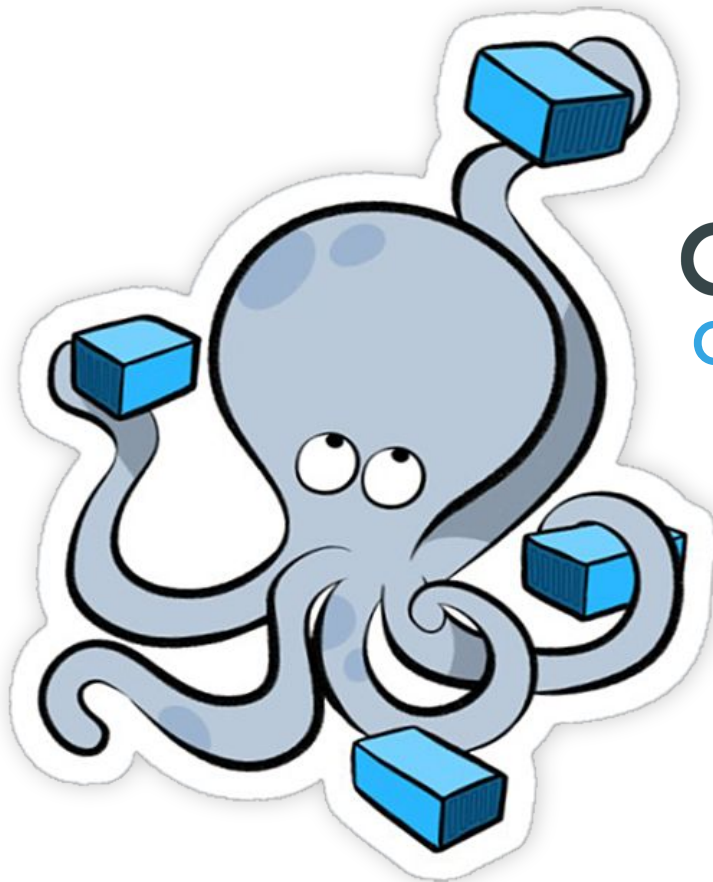


Scale

Many servers may use the same database.

As we saw in our last demo and activity, this was the case in our `docker-compose.yml` file.

We had multiple defined services, and the database was its own container `demo-db` in the demo and `db` in the activity.

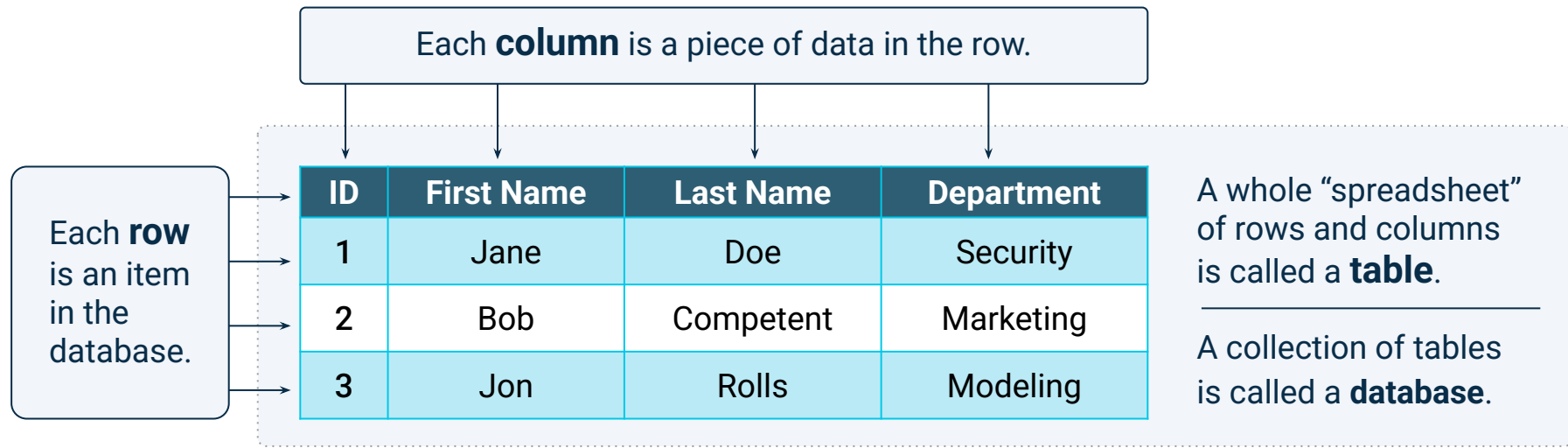


docker
compose

SQL Databases

While the two most important types of databases are SQL and NoSQL, we will focus only on SQL today.

SQL databases organize data like a spreadsheet.



Interacting with Data

Once you've created a table, there are four main ways to interact with the data it contains. Together, these are as known as **CRUD** operations.

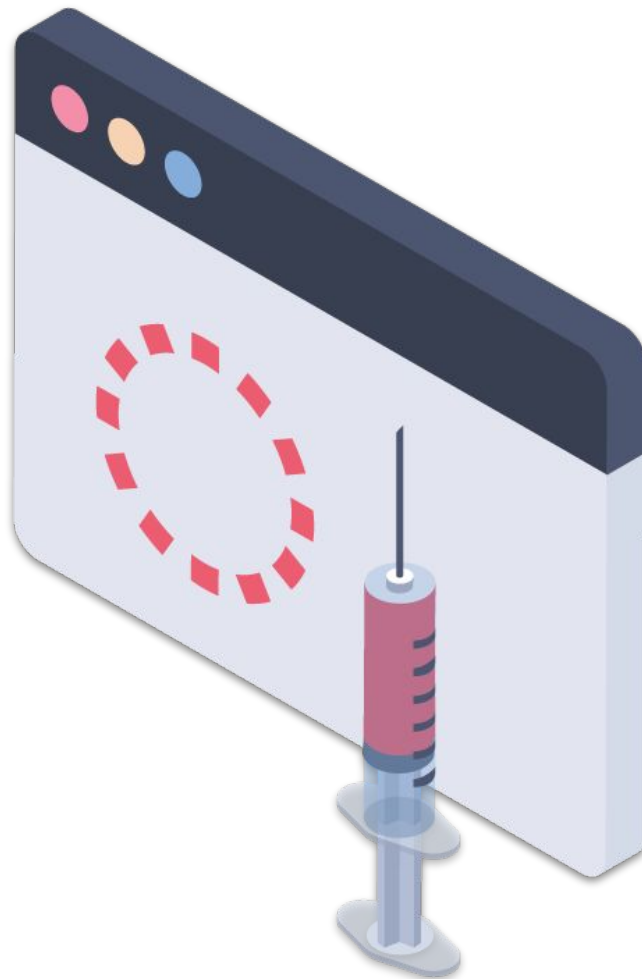
Create	Add new entries to a database.	When a user makes a new account in a web app, they are creating a new database entry with their user info.
Read	View entries in a database.	When a web application verifies your login credentials, it is reading the saved credentials it has in the database.
Update	Modify existing data in a database.	Whenever you need to reset your password in a web application, the password entry (more specifically, the hashed password) is updated with the new password.
Delete	Destroy data in the database.	If you have the option to delete your account from a web application (for example, on a GDPR web app), your account info will be deleted from the database.

CRUD

CRUD operations are also known as **queries**.

We'll use the term "query" to describe any input that will interact with data.

We'll come back to the SQL syntax of these operations to set the stage for SQL injections later in the course.





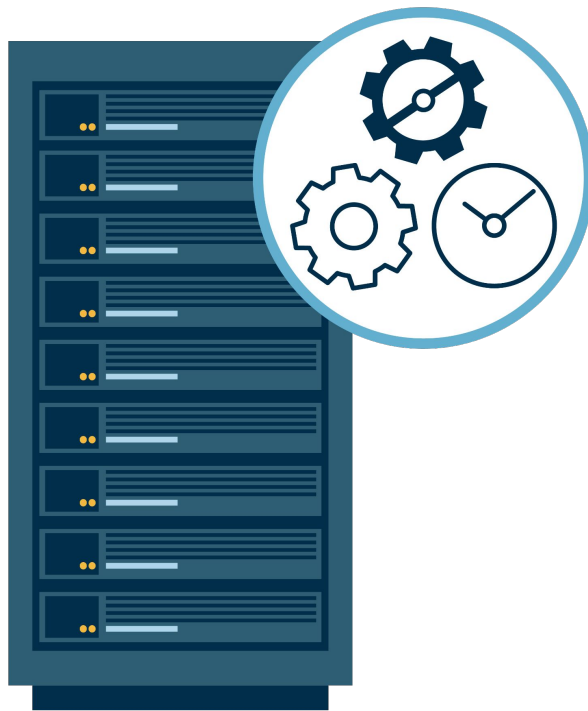
SQL injections are a type of web application attack intended to manipulate queries so that an attacker can use a database in ways the developers did not originally intend.

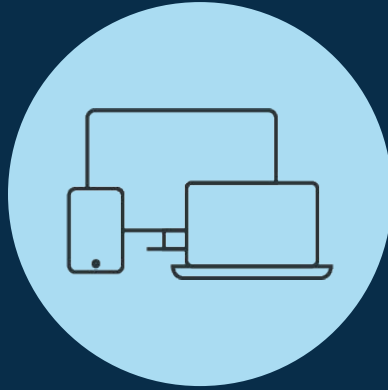


Demonstration Scenario

Let's see how we can create and enter queries into a web application's database to carry out basic database administration tasks.

- Deploy a web application.
- Navigate to the web application's page to see what data can be viewed by a web client.
- Start a bash session in the database's container and log into the database.
- Use a SQL query to retrieve the contents of the database's primary table.
- Modify that SQL query to find specific data.
- Add new data to the database.
- Delete data from the database.





Instructor Demonstration

Database



Activity:

Database Management

In this activity, students will play the role of a junior web engineer for the GoodCorp company and will manage its web application.

- GoodCorp uses Docker Compose and a set of containers to deploy and maintain its employee database website and application.
- They are tasked with locally deploying GoodCorp's employee directory website with Docker Compose. They also need to manage the data inside the employee directory database.

Suggested Time:
25 Minutes



Time's up!
Let's review

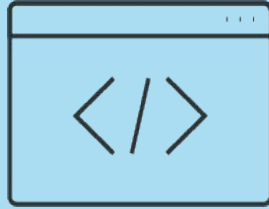


Questions?





Break
15 mins



Segue in SQL Injections



Let's look at some queries that may seem pointless or improperly created but are actually **fundamental** to how SQL injections work.



SQL Injections

What will happen when this query is run?

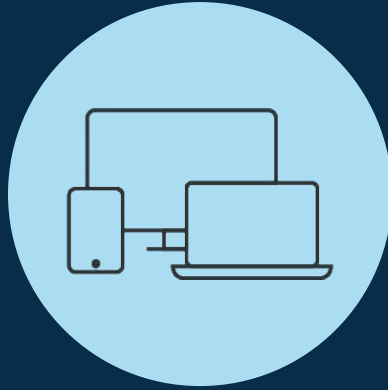
```
SELECT * FROM employees WHERE 1=1;
```



SQL Injections

```
SELECT * FROM employees WHERE 1=1;
```

The `1=1` part of the statement will evaluate to true since the number one always equals one, therefore, running the `SELECT * FROM employees` statement.



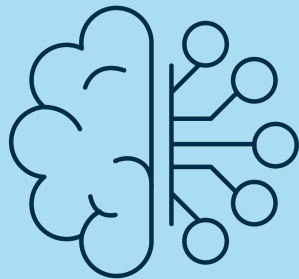
Instructor Demonstration

Segue in SQL Injections



Questions?





Using AI to Create a Web Application



We will now use **GitHub Copilot**, an AI application, to craft our own web application.

Using AI to Create a Web Application

Setting up GitHub Copilot involves a few steps to get everything running.

Prerequisites

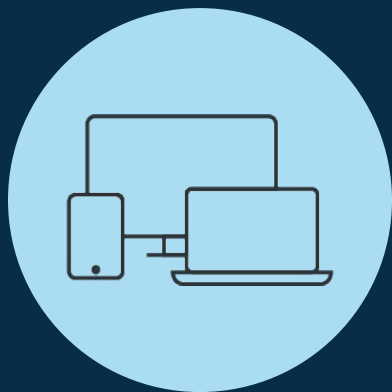
- Github Account
- Visual Studio Code

GitHub Copilot Extension

- Open VS Code
- Extension Marketplace
- Search for GitHub CoPilot
- Install Extension

Sign In to GitHub

- Activate Extension
- Authentication
- CoPilot Sign Up
- Allow Public Code



Instructor Demonstration

Using GitHub Copilot



Activity:

Creating a Website with Copilot

In this activity, students will continue to play the role of a junior web engineer for the GoodCorp company and will create a basic website.

- GoodCorp wants a basic front end that will have an HTML page, CSS styling, and a javascript file.

Suggested Time:
25 Minutes





Time's up!
Let's review



Questions?





Recap

Today's Lesson Covered:

01

Microservices

02

Stacks

03

Database management with websites

04

Developing a front-end website using Github CoPilot



Next class

Introduction to Web Vulnerabilities and Hardening



Thank you